Live timing load test

25 October 2026, 10:16–10:47 UTC · live.openskiti.me (Azure Container Apps, Sweden Central, 1 replica, 0.5 vCPU, 1 GiB) · OpenSkiTime desktop publishing through the live timing control panel · synthetic race data

One 0.5 vCPU server delivered every one of 309 updates of a 100-racer slalom to 900 simultaneous viewers with no failed connection during the race, using a fraction of its CPU and memory. After a forced restart all viewers were back within 12 seconds and nobody saw an empty page.

Simultaneous viewers	Failed or dropped during the race	Fan-out spread p95
900	0	267 ms
all connected at the first attempt	of 278,100 messages	after the first viewer of an update

Desktop to viewer p50	Back after restart p95	Peak server CPU in race
338 ms	7.4 s	39 %
includes the desktop's 250 ms cycle	all 900 viewers recovered	of the 0.5 vCPU allocation (5 s samples)

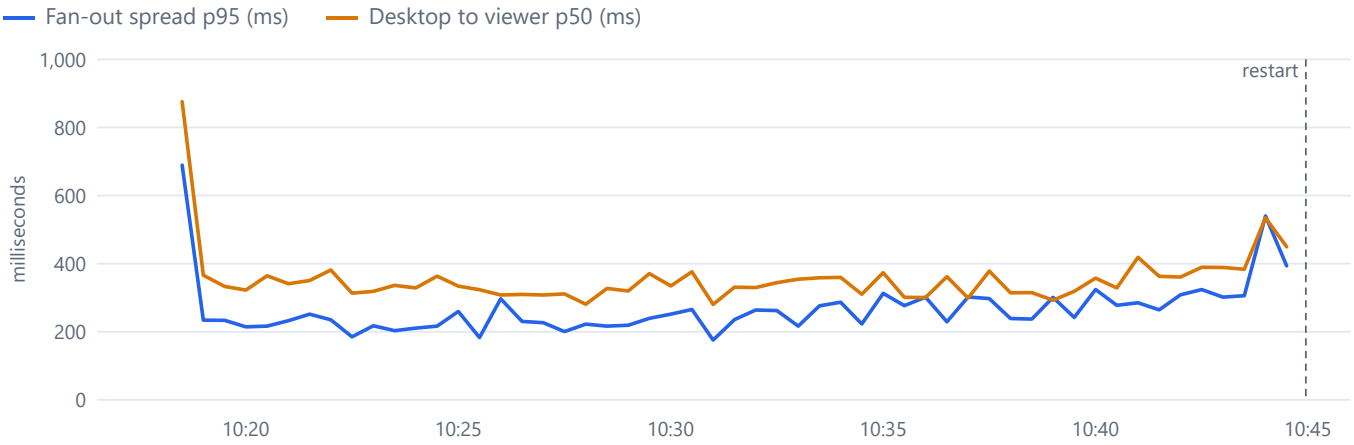
Setup

- **Publisher:** the real OpenSkiTime desktop application timed the 100-racer synthetic slalom in real time with the training simulator (a start every 15 s, run times 47–60 s) and published to Cloud through the live timing control panel (309 updates).
- **Viewers:** 900 simulated browsers in five Azure Container Instances (Sweden Central), 180 each from five different public addresses, so the production limit of 200 viewers per address applied unchanged. Each viewer loads the race page and its scripts and then follows the race over SignalR exactly like `live.js`: 15 s pings, a 35 s silence timeout and a jittered 0.5–15 s reconnect.
- **Server:** production configuration unchanged (1,000 connections, 200 per address, 3,000 requests per minute per address). A few real visitors opened the race from the public front page (up to 906 viewers).
- **Restart:** after the last finish the revision was restarted at 10:44:57 UTC to measure recovery.

Delivery

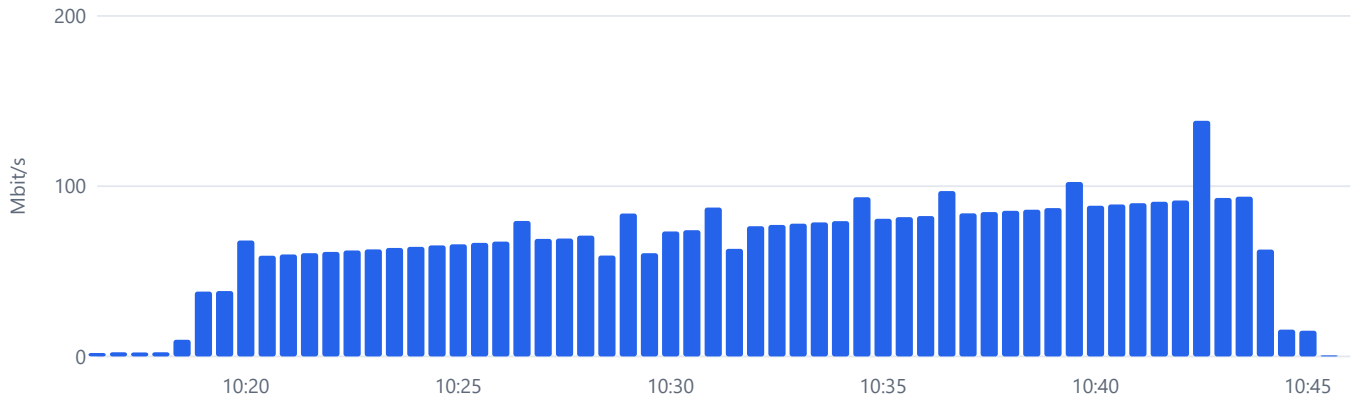
Delivery delay per 30 s

Fan-out spread: how much later than the first viewer the others received the same update. Desktop to viewer: from the publishing computer's stamp, corrected for its measured clock offset of +0.72 s. Both grow with the results table.



Traffic to all viewers per 30 s

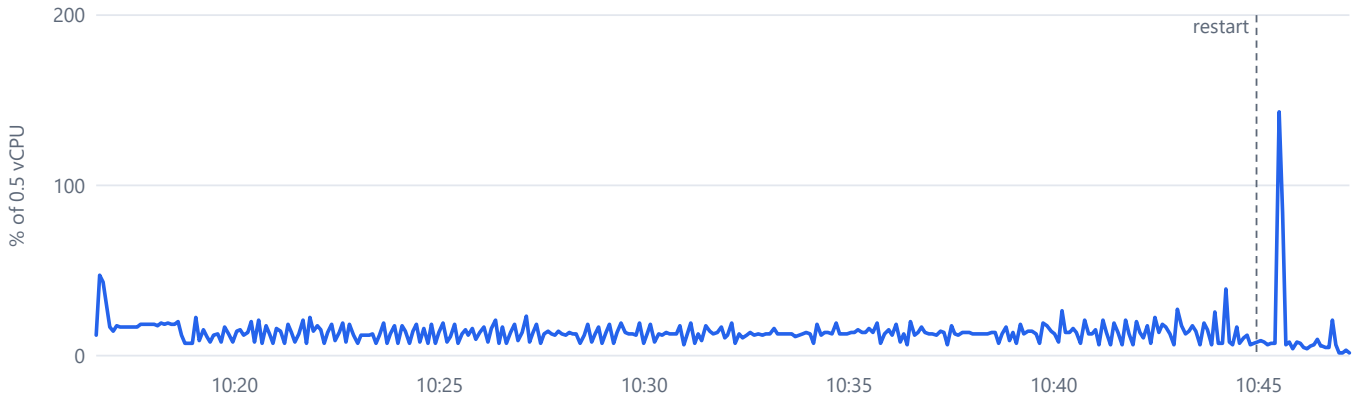
Every update is a full snapshot (40–66 kB) sent to every viewer, so traffic grows with viewers × snapshot size × updates.



Server

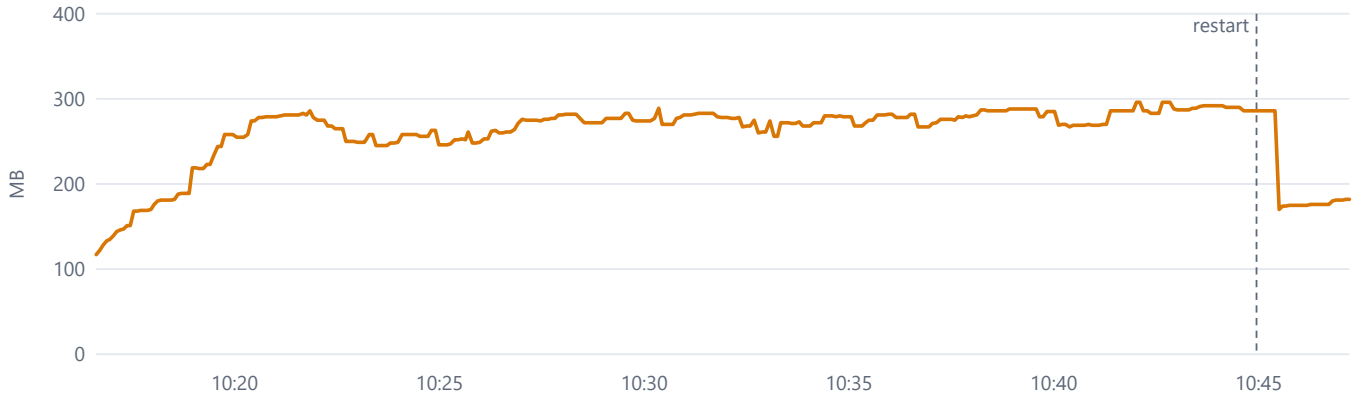
Server CPU (control panel Server load, every 5 s)

CPU converted to a share of the 0.5 vCPU allocation (the panel shows a share of the 4 cores .NET sees). The spike is 900 viewers reconnecting to the freshly started replica.



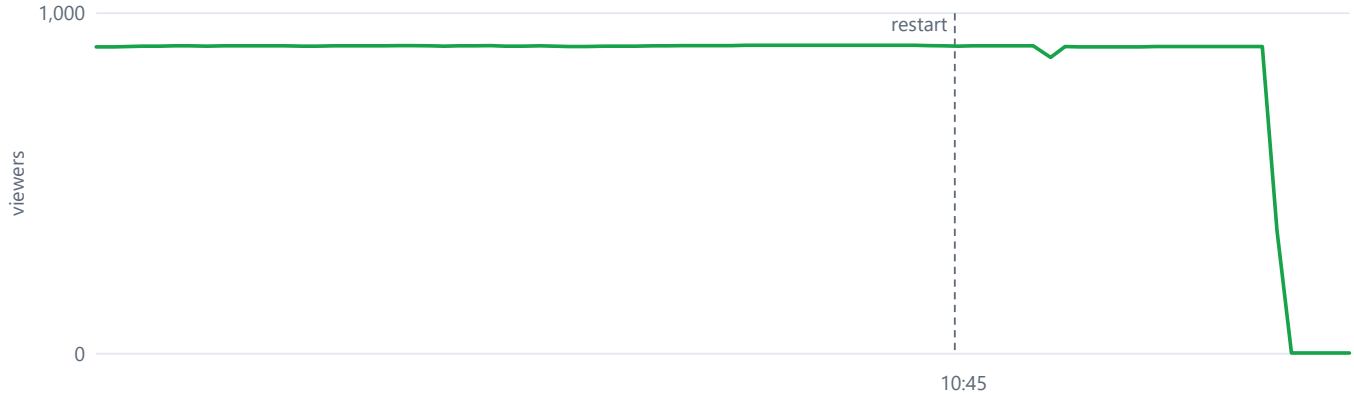
Server memory (working set, every 5 s)

Grows with the results table and the viewer connections; the restart starts a fresh process.



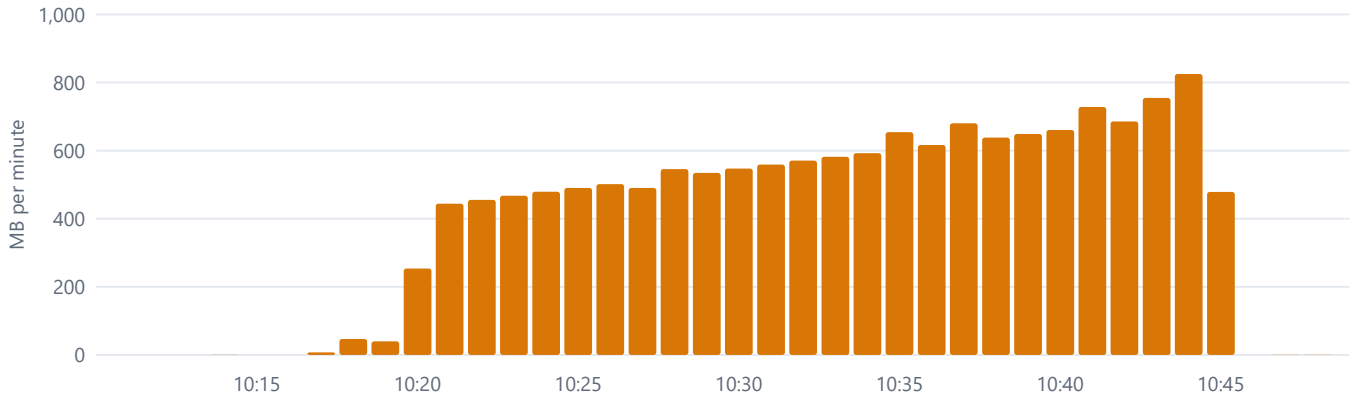
Viewers through the restart

Viewers counted by the server. The new replica started at 10:44:57; all old connections closed at 10:45:19 and were back within seconds. The drop at the end is the load generators stopping.



Network out per minute (Azure Monitor)

Platform view of the same container: up to 825 MB a minute (110 Mbit/s on average over that minute), 15.0 GB in total. CPU peaked at 104 millicores a minute (21 % of 500).



Measured distributions

Measure	Area	Samples	p50	p95	Max	Unit
Connect to first state	Connections	900	8	15	327	ms
Fan-out spread after first viewer	Delivery	278,100	101	267	700	ms
Slowest viewer per update	Delivery	309	238	338	700	ms
Desktop publish to viewer	Delivery	278,100	338	529	1,181	ms
State message size	Delivery	278,100	53	64	66	kB
Restart: drop to next state	Restart	901	1,508	7,442	11,874	ms

Combined over all five load generators. A local rehearsal on one computer (no network) gave a desktop-to-viewer p50 of 270 ms, so most of that delay is the desktop's 250 ms live snapshot cycle.

Slowest updates

Version	Viewers reached	Slowest after first (ms)	Size (kB)
3	900	700	39.4
309	900	621	65.5
257	900	523	61.4
308	900	496	65.3
86	900	410	46.7
311	900	407	65.6

What it means

- **900 viewers fit comfortably on one replica.** CPU averaged about 13 % and peaked at about 40 % of the allocation during the race; memory stayed under 300 MB of 1 GiB. The limits that bind first are settings: 1,000 connections in total and 200 per client address, which a shared venue Wi-Fi reaches first.
- **Fan-out time grows with viewers × snapshot size.** The load generators used only 25–54 of their 750 millicores and every generator's first viewer received an update within about 1 ms of the first viewer overall, so the 100–700 ms spread arises at the server and its ingress (about 48 MB of writes per update).
- **Bandwidth is the figure to watch for bigger events.** One viewer following a whole run used about 15 MB. Larger fields make every snapshot larger; sending only changed rows would cut traffic many times over.
- **Restarts are handled well.** The control panel restored the race on the new replica before the viewers returned, so no viewer received an empty state. One viewer connected to the draining replica was recovered by the 35 s silence timeout.
- **The panel's CPU figure** is a share of the 4 cores .NET sees, not of the 0.5 vCPU allocation: multiply by eight. Container Apps let the process burst to 0.72 cores while 900 viewers reconnected.

Limits of the test

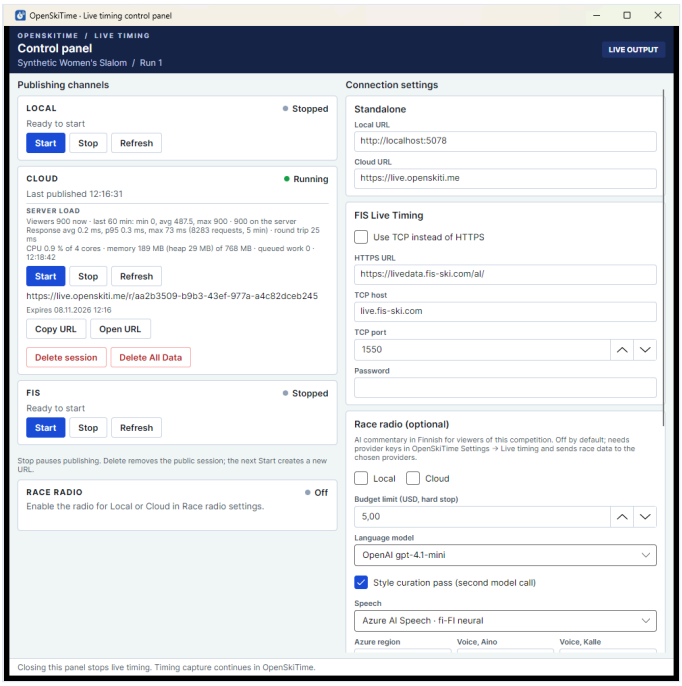
- One race session, no race radio, fields of 100 racers only.
- Viewers were protocol-level clients in Azure, not browsers on mobile networks.
- Desktop-to-viewer delay depends on the publishing computer's clock, measured against time.windows.com before and after (−0.703 s and −0.728 s).

Cost

The test cost well under one euro (five Container Instances for about 75 minutes, a Basic container registry and a storage account for a few hours, all deleted afterwards; the live server itself stays within the Container Apps monthly free grant). In normal use the live server costs nothing between races (scale to zero); per race the main variable cost is outbound traffic beyond the first 100 GB a month.

The operator's view

900 viewers connected, before the first start



After the server restart

